Category 2: Networking and Content Delivery 🌐

If IAM is the bouncer checking IDs, Networking is the physical walls, doors, and hallways of your cloud. Understanding the Virtual Private Cloud (VPC) is the most critical hurdle for any AWS DevOps engineer.

☁️ Amazon VPC (Virtual Private Cloud) MUST KNOW

What it is: A logically isolated section of the AWS Cloud where you can launch AWS resources in a virtual network that you define.

Analogy: Fencing off your own private plot of land in the massive AWS public park. Only people you allow through the gate can get in.

Why and when to use it: Required for almost everything. It provides absolute control over your virtual networking environment, including resource placement, connectivity, and security.

Key concepts: CIDR blocks (e.g., `10.0.0.0/16` defines the IP range), Region-bound (a VPC spans all AZs in a single region).

How it works: It acts as the outer container. Every EC2 instance, RDS database, or ECS cluster you deploy must live inside a VPC.

Common example: Creating a custom VPC for production workloads with strict IP routing rules, completely separate from your development VPC.

Pricing model: The VPC itself is free. You pay for certain components attached to it (like NAT Gateways) and data transfer out to the internet.

Best practices and pitfalls: *Pitfall:* Using overlapping CIDR blocks across different VPCs (e.g., both are `10.0.0.0/16`). If you ever need to connect them later via VPC Peering, they will conflict. Always plan your IP ranges!

Note: 🎯 Exam Focus: Know that the default VPC exists just so beginners can launch EC2s immediately, but you should always create custom VPCs for real workloads.

Related services: Subnets, Route Tables, Internet Gateway, EC2, RDS.

🗂️ Subnets MUST KNOW

What it is: A smaller, segmented chunk of IP addresses within your VPC, tied to a specific Availability Zone (AZ).

Analogy: The individual rooms within your fenced-off plot of land (the VPC). Some rooms have

🗺️ Route Tables MUST KNOW

What it is: A set of rules, called routes, that determine where network traffic from your subnet or gateway is directed.

Analogy: The GPS navigation system or signposts at intersections telling packets which wire to go down.

Why and when to use it: To connect subnets to the internet, to other VPCs, or to on-premise networks.

Key concepts: Main Route Table (default for VPC), Custom Route Table (explicitly associated with a subnet), Destination (where traffic wants to go), Target (the gateway/connection handling it).

How it works: You associate a Route Table with a Subnet. If a packet tries to reach `0.0.0.0/0` (the internet), the table looks for a target (like an Internet Gateway) to send it to.

Common example: A Public Route Table pointing `0.0.0.0/0` to an IGW. A Private Route Table pointing `0.0.0.0/0` to a NAT Gateway.

Pricing model: Free.

Best practices and pitfalls: *Pitfall:* Forgetting to explicitly associate a custom route table with your subnet, causing it to fall back to the restrictive default main route table.

Note: The "local" route (routing traffic within the VPC CIDR) is built-in and cannot be deleted.

Related services: Subnets, IGW, NAT Gateway.

🚪 Internet Gateway (IGW) MUST KNOW

What it is: A horizontally scaled, redundant, and highly available VPC component that allows communication between your VPC and the internet.

Analogy: The main front door of your building that leads out to the public street.

Why and when to use it: When resources inside your VPC (like an ALB or bastion host) need to be accessible from the outside world, or need to reach out to the world directly.

Key concepts: Attached to a VPC (1:1 relationship).

How it works: 1. Create IGW. 2. Attach to VPC. 3. Update Route Table to point internet traffic (`0.0.0.0/0`) to the IGW.

Common example: Making a web server reachable via a public IP address.

Pricing model: Free.

Best practices and pitfalls: It is fully managed; you never have to worry about its bandwidth or

🔒 VPC Security: SGs vs NACLs

🧱 Security Groups (SG) MUST KNOW

What it is: A virtual stateful firewall at the *instance* level to control inbound and outbound traffic.

Analogy: The security guard standing directly outside an employee's specific office door.

Key concepts: Stateful (if request is allowed in, the response is automatically allowed out). Default is DENY all inbound, ALLOW all outbound.

Common example: Allowing port 443 (HTTPS) from `0.0.0.0/0`, and port 22 (SSH) only from your VPN IP.

🚧 Network Access Control Lists (NACL) SHOULD KNOW

What it is: A virtual stateless firewall at the *subnet* level.

Analogy: The security guard at the front gate of the building. Traffic must pass this before getting to the office doors.

Key concepts: Stateless (must explicitly define BOTH inbound and outbound rules). Evaluated in number order (lowest number rule wins).

Common example: Blocking a specific malicious IP address from even entering the subnet.

⚖️ Comparison: Security Group vs NACL

Feature	Security Group (SG)	NACL
Scope	Instance / Resource Level (e.g., EC2, RDS)	Subnet Level
State	Stateful (return traffic automatically allowed)	Stateless (must configure return rules)
Rules	ALLOW rules only (implicit deny all else)	ALLOW and DENY rules
Evaluation	All rules evaluated before deciding	Evaluated in order (lowest number first)

🤝 VPC Peering SHOULD KNOW

What it is: A networking connection between two VPCs that enables you to route traffic between them using private IPv4/IPv6 addresses.

Analogy: Running a private ethernet cable directly between two different office buildings.

🧭 Amazon Route 53 MUST KNOW

What it is: A highly available and scalable cloud Domain Name System (DNS) web service.

Analogy: The phonebook of the internet. You look up "Bob" (google.com) and it gives you his phone number (IP address).

Why and when to use it: For registering domain names, routing internet traffic to your ALB or S3 buckets, and checking the health of your resources.

Key concepts: Hosted Zones (container for DNS records), A Records (Name to IP), CNAME (Name to Name), Alias Records (AWS specific: Name to AWS Resource).

How it works: Users type your URL -> Route 53 looks up the routing policy -> Returns the IP of your ALB.

Common example: Using "Latency-based routing" to send European users to your Frankfurt ALB and US users to your Virginia ALB.

Pricing model: ~$0.50/month per Hosted Zone, plus pennies per million queries.

Best practices and pitfalls: Always use AWS ALIAS records instead of CNAMEs when pointing a root domain (e.g., `myapp.com`) to an AWS resource (like an ALB), as DNS RFCs forbid CNAMEs at the root level.

Note: 🎯 Exam Focus: Only Route 53 provides a 100% SLA (Service Level Agreement) in AWS. It never goes down.

Related services: CloudFront, ALB, S3.

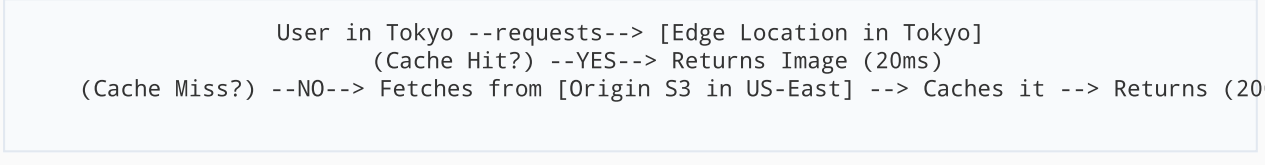
⚡ Amazon CloudFront MUST KNOW

What it is: A fast Content Delivery Network (CDN) service that securely delivers data, videos, applications, and APIs to customers globally.

Analogy: Stocking your product in local retail stores globally, rather than shipping every single order from one warehouse in New York.

Why and when to use it: To drastically reduce latency for users by caching static assets (HTML/CSS/JS/Images) at Edge Locations closest to them.

Key concepts: Distribution (the CDN configuration), Origin (where the CDN gets the original file, e.g., an S3 bucket or ALB), Edge Location (the mini data center holding the cache).



Category 3: Storage 💾

Compute is fleeting, but data must persist. AWS provides different storage tools depending on whether you need to store files, mount drives to servers, or query databases.

🪣 Amazon S3 (Simple Storage Service) MUST KNOW

What it is: An object storage service offering industry-leading scalability, data availability, security, and performance.

Analogy: An infinite digital warehouse with a valet. You hand them a box (file), they give you a ticket (URL), and they store it somewhere. You don't know or care exactly which shelf it's on.

Why and when to use it: Storing backups, static website assets, user uploads (images/videos), data lakes. It is the backbone of storage on AWS.

Key concepts:

- **Buckets:** The root-level folders. Bucket names must be GLOBALLY unique across all AWS accounts.
- **Objects:** The files themselves (keys and values). Max object size is 5TB.
- **Versioning:** Keeps multiple variants of an object in the same bucket (prevents accidental deletion).

How it works: It is a flat architecture, not a file system. Folders are an illusion created by the key name (e.g., `images/2023/pic.jpg` is just one long filename string).

Common example: Storing Terraform state files, or serving images for a social media app.

S3 Storage Classes & Lifecycle Policies

S3 is priced on storage tiers. A **Lifecycle Policy** automatically moves older data to cheaper tiers.

Class	Use Case	Cost Profile
S3 Standard	Active, frequently accessed data (images on an active app).	Highest storage cost, low retrieval.
S3 Standard-IA	Infrequently Accessed. Data accessed < once a month.	Lower storage cost, pay per retrieval.
S3 Glacier Flexible	Archiving backups. Retrieval takes minutes to hours.	Very cheap storage, retrieval fee.
S3 Glacier Deep Archive	Regulatory compliance data kept for 10 years. Retrieval takes 12+ hours.	Lowest storage cost (pennies), high retrieval fee.

Pricing model: Pay for storage per GB, plus PUT/GET requests, plus Data Transfer Out.

Best practices and pitfalls: *Pitfall:* Making a bucket Public. By default, all buckets are private. Turn

💽 Amazon EBS (Elastic Block Store) MUST KNOW

What it is: High-performance block storage volumes designed for use with EC2 instances.

Analogy: A virtual USB thumb drive or internal hard drive that you plug directly into your virtual computer (EC2).

Why and when to use it: When your EC2 instance needs a persistent root operating system drive, or you need to run a relational database directly on EC2 requiring fast read/write speeds.

Key concepts: Volume types (gp3 for general purpose SSD, io2 for extreme IOPS databases). Snapshots (point-in-time backups of the volume saved to S3).

How it works: You provision a 50GB gp3 volume and attach it to an EC2 instance. It appears as `/dev/xvda`. You must format it (ext4/xfs) and mount it.

Common example: The C:\ drive or root `/` partition of your EC2 web server.

Pricing model: Billed per GB *provisioned*, not used. If you provision a 1TB drive but only put 1MB of data on it, you pay for 1TB.

Best practices and pitfalls: An EBS volume is locked to a single Availability Zone. If an EC2 fails in AZ-A, you cannot attach its EBS volume to an EC2 in AZ-B (unless you take a Snapshot first).

Note: If you terminate an EC2 instance, the root EBS volume is deleted by default unless you uncheck "Delete on Termination".

Related services: EC2, KMS (for encryption).

📁 Amazon EFS (Elastic File System) SHOULD KNOW

What it is: A serverless, fully elastic file storage system that allows multiple EC2/ECS instances to mount it concurrently.

Analogy: A shared network drive (like a NAS) that everyone in the office can connect to at the same time and edit files.

Why and when to use it: When you have an Auto Scaling Group of 10 web servers that all need to read and write to the exact same directory (e.g., a shared WordPress `/wp-content/uploads` folder).

Key concepts: Uses NFS (Network File System) protocol. Multi-AZ (data is stored across multiple AZs automatically).

How it works: Create an EFS -> it creates Mount Targets in your subnets -> your EC2 instances mount it using the NFS client.

Common example: Providing shared persistent storage to Docker containers running on ECS Fargate.

⚖️ Storage Comparisons

Choosing the right storage is a classic architecture interview question. Memorize these differences.

EBS vs. EFS vs. S3

Feature	EBS (Block Storage)	EFS (File Storage)	S3 (Object Storage)
Access Pattern	1 EC2 instance at a time (usually)	1,000s of EC2/ECS instances concurrently	Millions of internet clients concurrently via HTTP API
Scope (AZ bounds)	Locked to a single AZ	Spans multiple AZs in a Region	Spans multiple AZs in a Region (Globally unique name)
Performance	Ultra-low latency, high IOPS. Best for DBs.	Low latency, high throughput. Best for shared files.	Higher latency. Best for large files, media, backups.
Scaling & Payment	Provision size upfront (pay for provisioned).	Auto-scales (pay for exactly what is used).	Infinite scale (pay for exactly what is used).
OS Visibility	Appears as a raw drive (needs formatting).	Appears as a network mounted directory.	Accessed via API/SDK, not a mounted drive.

📝 **DevOps Practice Lab 3 (Storage):** Using Terraform, create an S3 bucket with versioning enabled. Upload a dummy `state.json` file. Create a lifecycle rule that transitions older versions of the file to Glacier after 30 days. Next, provision a small EC2 instance and attach a secondary 5GB EBS volume to it; SSH in, format the drive, and mount it to `/data`.

💡 **Architectural Tip:** When building modern CI/CD pipelines, you will push your Docker images to **ECR**, store your Terraform State in **S3**, and mount **EBS** or **EFS** to your running **ECS/EKS** nodes depending on statefulness.